

Questions:

Objective

To understand and implement the core Object-Oriented Programming (OOP) concepts in Java. Concepts Covered

- Classes and Objects
- Constructors
- Encapsulation
- Inheritance
- Method Overloading
- Method Overriding
- Polymorphism
- Abstraction
- Interfaces

Task Description

Develop a "**Library Management System**" to demonstrate Object-Oriented Programming concepts.

Part A: Classes and Objects

Create a class named Book containing:

- Book ID
- Book Name
- Author
- Price

Create objects and display their details.

Part B: Constructors and Encapsulation

- Create both default and parameterized constructors.
- Declare data members as private.
- Access them using Getter and Setter methods.

Part C: Inheritance

Create the following classes:

- Person
- Student (inherits Person)
- Faculty (inherits Person)

Display the inherited properties using objects.

Part D: Polymorphism

Implement:

- Method Overloading for calculating the area of different shapes.
- Method Overriding by creating a Vehicle class and overriding the display() method in Car and Bike classes.

Part E: Abstraction and Interfaces

- Create an abstract class Shape containing an abstract method draw().

- Implement the draw() method in Circle and Rectangle classes.
- Create an interface Printable and implement it in a class named Report.

Step wise execution

Step 1: Start the Java program.

Step 2: Create the Book class with Book ID, Book Name, Author and Price.

Step 3: Declare Book data members as private.

Step 4: Create a default constructor and a parameterized constructor.

Step 5: Create getter and setter methods for accessing and modifying private data.

Step 6: Create Book objects and display their details.

Step 7: Create a Person class containing common properties such as name and age.

Step 8: Create Student and Faculty classes by extending Person.

Step 9: Create Student and Faculty objects and display inherited properties.

Step 10: Create the Area class and overload calculateArea() for different shapes.

Step 11: Create Vehicle as the parent class.

Step 12: Create Car and Bike classes that override the display() method.

Step 13: Create Vehicle reference variables and assign Car and Bike objects to demonstrate runtime polymorphism.

Step 14: Create the abstract Shape class with abstract draw() method.

Step 15: Create Circle and Rectangle classes and implement draw().

Step 16: Create Printable interface with print() method.

Step 17: Implement Printable in the Report class.

Step 18: Create Report object and call print().

Step 19: Execute all operations from main().

Step 20: Stop the program.

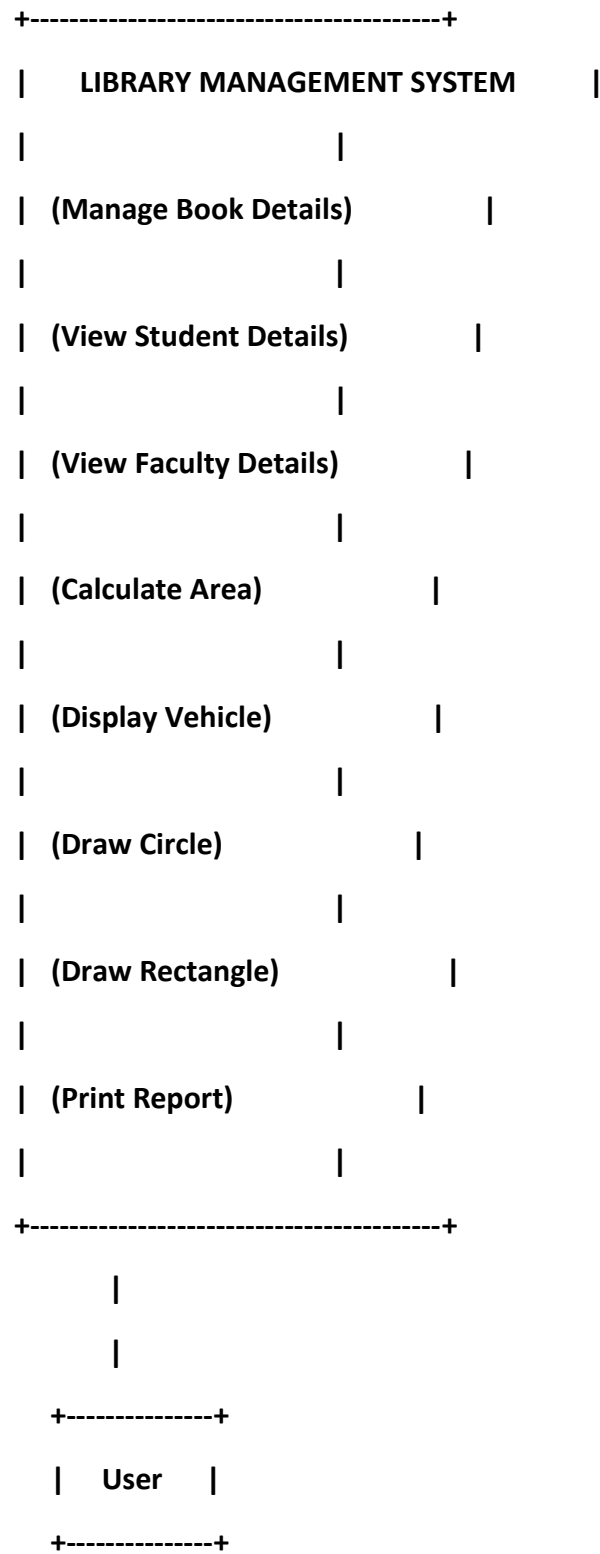
Algorithm:

1. Start.
2. Create the Book class.
3. Declare bookId, bookName, author and price as private.
4. Create default and parameterized constructors.

5. Create getter and setter methods.
6. Create Book objects and display book details.
7. Create Person class with name and age.
8. Create Student class extending Person.
9. Create Faculty class extending Person.
10. Create Student and Faculty objects.
11. Display inherited properties.
12. Create Area class.
13. Define calculateArea() methods with different parameters:
 - a. Circle
 - b. Rectangle
 - c. Triangle
14. Call the overloaded methods and display the results.
15. Create Vehicle class with display() method.
16. Create Car and Bike classes extending Vehicle.
17. Override display() in Car and Bike.
18. Create Vehicle references for Car and Bike.
19. Call display() to demonstrate runtime polymorphism.
20. Create abstract Shape class with draw() method.
21. Create Circle and Rectangle classes.
22. Implement draw() in both classes.
23. Create objects using Shape references.
24. Call draw().
25. Create Printable interface with print() method.
26. Create Report class implementing Printable.
27. Implement print() in Report.
28. Create Report object and call print report

29. Stop.

UML diagram



Code:

```
package basics;

class Book {
    private int bookId;
    private String bookname;
    private String author;
    private int price;

    public Book(int id, String name, String author, double
price) {
        this.bookId = id;
        this.bookname = name;
        this.author = author;
        this.price = (int) price;
    }

    public int getBookId() { return bookId; }
    public void setBookId(int bookId) { this.bookId = bookId; }

    public String getBookname() { return bookname; }
    public void setBookname(String bookname) { this.bookname =
bookname; }

    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author =
author; }

    public int getPrice() { return price; }
    public void setPrice(int price) { this.price = price; }

    public void displayDetails() {
        System.out.println(bookId + " " + bookname + " " +
author + " " + price);
    }
}
```

```

}

class Person {
    String name;
    void display() {
        System.out.println("Name: " + name);
    }
}

class Student extends Person {
    String course;
    void display() {
        super.display();
        System.out.println("Course: " + course);
    }
}

class Faculty extends Person {
    String department;
    void display() {
        super.display();
        System.out.println("Department: " + department);
    }
}

// Polymorphism
class AreaCalculator {
    double area(double radius) {
        return 3.14 * radius * radius;
    }
    double area(double l, double b) {
        return l * b;
    }
    double area(double bs, double h, boolean tri) {
        return 0.5 * bs * h;
    }
}

```

```
}

class Vehicle {
    void display() {
        System.out.println("This is a vehicle");
    }
}

class Car extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a car");
    }
}

class Bike extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a bike");
    }
}

abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("This is a circle");
    }
}

class Rectangle extends Shape {
    @Override
    void draw() {
```



```

        System.out.println("Draw a rectangle");
    }
}

interface Printable {
    void print();
}

class Report implements Printable {
    public void print() {
        System.out.println("Printing report...");
    }
}

public class LibraryManagementSystem {
    public static void main(String[] args) {
        System.out.println("Book Details:");
        Book b = new Book(123, "Java Programming", "Santosh
Kumar P", 500);
        b.displayDetails();

        System.out.println("In Inheritance: ");
        Student s = new Student();
        s.name = "Prabhash";
        s.course = "AIML";
        s.display();

        Faculty f = new Faculty();
        f.name = "Dr. Santosh";
        f.department = "Engineering";
        f.display();

        System.out.println("In Polymorphism: ");
        AreaCalculator ac = new AreaCalculator();
        System.out.println("Circle area: " + ac.area(10));
    }
}

```

```
        System.out.println("Rectangle area: " + ac.area(6,
10));
        System.out.println("Triangle area: " + ac.area(3, 7,
true));

        Vehicle v = new Vehicle();
        Vehicle car = new Car();
        Vehicle bike = new Bike();
        v.display();
        car.display();
        bike.display();

        System.out.println("Abstraction & Interfaces: ");
        Shape circle = new Circle();
        Shape rectangle = new Rectangle();
        circle.draw();
        rectangle.draw();

        Report report = new Report();
        report.print();
    }
}
```

Output screen shot:

The screenshot shows an IDE with a dark theme. The menu bar at the top includes File, Edit, Selection, View, Go, Run, Terminal, and Help. The toolbar below the menu bar contains icons for file operations and running. The sidebar on the left has icons for Problems, Output, Debug Console, Terminal, and Ports. The main editor area shows a Java file named 'bas.java'. The terminal window at the bottom displays the output of a Java application, including class details, book details, and a report. The output is as follows:

```

This is a car
This is a bike
Abstraction & Interfaces:
This is a circle
Draw a rectangle
Printing report...
PS C:\Users\Admin\OneDrive\Documents\basicjava> ^C
PS C:\Users\Admin\OneDrive\Documents\basicjava>
PS C:\Users\Admin\OneDrive\Documents\basicjava> c;; cd 'C:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '-enable-preview' '-XX:showCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'oops.LibraryManagementSystem'
Book Details:
223 Java Programming Santosh Kumar P 500.0
In Inheritance:
Name: Prabhash
Course: ADML
Name: Dr. Santosh
Department: Engineering
In Polymorphism:
Circle area: 314.0
Rectangle area: 60.0
Triangle area: 10.5
This is a vehicle
This is a car
This is a bike
Abstraction & Interfaces:
This is a circle
Draw a rectangle
Printing report...
PS C:\Users\Admin\OneDrive\Documents\basicjava>
  
```